Python Imports

How `import` finds, loads, caches and binds modules – and the forms of the statement.

Import forms

Statement	Binds	Then use
`import math`	`math`	`math.sqrt(4)`
`import numpy as np`	alias `np`	`np.array(...)`
`from math import sqrt`	`sqrt` only	`sqrt(4)`
`from math import pi as PI`	alias `PI`	`PI`
`import os.path`	`os`	`os.path.join(...)`
`from . import x`	relative	inside packages
`from m import *`	all public names	**avoid** – pollutes namespace

```
import math
from math import sqrt, pi as PI
import os.path as osp

print(math.floor(2.7), sqrt(16), round(PI, 2))
print(osp.basename("/a/b/c.txt"))
```

```
OUTPUT
2 4.0 3.14
c.txt
```

How Python finds a module

- **`sys.modules` cache** – already imported? Reuse it; module code runs only once.
- **Built-in / frozen** modules compiled into Python.
- **Each entry of `sys.path`, in order**: script folder (or cwd), `PYTHONPATH`, standard library, `site-packages`.

```
import sys
print("math" in sys.modules, type(sys.path).__name__)
import math as m2
# same cached module object
print(m2 is math)
```

```
OUTPUT
True list
True
```

PITFALL First match wins: a file called `random.py` or `json.py` in your project shadows the standard-library module.

Where did that module come from?

```
import json
print(json.__name__)
# a package
print(json.__file__.endswith("__init__.py"))
```

```
OUTPUT
json
True
```

- `mod.__file__` / `mod.__spec__` show the source and loader.
- `python -X importtime app.py` reports slow imports.
- `python -c "import sys; print(sys.path)"` shows the search path.

Dynamic imports & reload

```
import importlib
# import by string
mod = importlib.import_module("json")
print(mod.dumps({"a": 1}))
# re-run module code
importlib.reload(mod)
```

```
OUTPUT
{"a": 1}
```

Optional dependencies

```
try:
    import not_a_real_module
except ModuleNotFoundError as e:
    print("missing:", e.name)
```

```
OUTPUT
missing: not_a_real_module
```

Circular imports

`a` imports `b` while `b` imports `a` → half-initialised module or `ImportError`. Fixes:

- Move shared code into a third module.
- Import inside the function that needs it (lazy import).
- `import b` and use `b.name` later, rather than `from b import name`.

```
from typing import TYPE_CHECKING
if TYPE_CHECKING:                # False at runtime
    # only for type checkers
    from shop.cart import Cart

def total(cart: "Cart") -> float: ...
```

PEP 8 import style

- Imports at the **top** of the file, one module per line.
- Order: standard library → third-party → local, with a blank line between groups.
- Prefer absolute imports; avoid wildcard `*` imports.